Reflexión individual - Saúl Gabriel Nolasco Cen

En el desarrollo de software real, el rendimiento de una aplicación no depende únicamente del hardware en el que se ejecuta, sino de la eficiencia de cómo se procesan los datos.

En esta problemática, trabajamos con un archivo de ejemplo de 262,144 registros. Procesar tanta información con algoritmos ineficientes podría congelar la ejecución del programa o hacer inviable su uso ya que a un usuario no le gustaría esperar mucho tiempo. Para dimensionar esto en tiempo real, se toma como referencia técnica que una computadora promedio procesa alrededor de 10^8 operaciones básicas por segundo.

Si para resolver este problema hubiéramos utilizado un algoritmo de ordenamiento tradicional como Bubble Sort, cuya complejidad temporal en el peor caso es $O(N^2)$, el procesador habría tenido que realizar alrededor de $(262{,}144)^2 \approx 6.87 \times 10^{10}$ operaciones. Dividiendo esta cantidad entre la capacidad de procesamiento de la computadora (10^8 operaciones/seg), el programa habría tardado aproximadamente 11.4 minutos solo en ordenar el archivo.

En cambio, al implementar Merge Sort, reducimos la complejidad a $O(N \log_2 N)$. Con $262{,}144$ registros, $\log_2(262{,}144) = 18$, lo que significa que el algoritmo realiza $262{,}144 \times 18 \approx 4.71 \times 10^6$ operaciones. Por lo tanto, Merge Sort completa la tarea en 0.047 segundos, pasando de minutos de espera a una ejecución casi instantánea.

Una vez que logramos tener la información ordenada, podemos optimizar la búsqueda de información. Si el arreglo estuviera desordenado, la única opción sería realizar una búsqueda secuencial comprobando registro por registro, lo que en el peor escenario tomaría $262{,}144$ comparaciones por cada consulta. Sin embargo, al tener los datos estructurados cronológicamente, pudimos implementar una búsqueda binaria. Este algoritmo tiene una complejidad $O(\log_2 N)$, lo que significa que para encontrar el primer registro del rango consultado solo se requieren un máximo de 18 comparaciones.

Esta actividad nos demuestra que implementar algoritmos eficientes es importante porque convierte lo que pudo ser un proceso lento de varios minutos en una consulta inmediata para el usuario.